Assignment No.: 5

Title:

Practical based on Aggregate Functions – AVG, COUNT, MAX, MIN, SUM, etc. using AdventureWorks2019 Database

Objectives:

1. To understand and use aggregate functions in SQL Server.
2. To summarize and analyse data from large database tables.
3. To apply functions such as AVG, COUNT, MAX, MIN, andSUM for report generation.
4. To practice data grouping and aggregation without using joins.

Problem Statement:

Write SQL queries to demonstrate the use of aggregate functions such as AVG, COUNT, MAX, MIN, and SUM using the **AdventureWorks2019** database tables — without using joins.

Database Tables Used:

1. **Production.Product** – Contains product information such as ListPrice and StandardCost.
2. **Sales.SalesOrderDetail** – Contains details of order quantity, line total, and unit price.
3. **Sales.SalesOrderHeader** – Contains information about each order such as OrderDate, SubTotal, TaxAmt, and TotalDue.

SQL Queries:

1) Find the total number of products available.

```
SELECT COUNT(ProductID) AS Total_Products
    FROM Production.Product;
```

2) Find the average, highest, and lowest list price of all products.

```
SELECT
    AVG(ListPrice) AS Average_Price,
    MAX(ListPrice) AS Highest_Price,
    MIN(ListPrice) AS Lowest_Price
FROM Production.Product
WHERE ListPrice > 0;
```

3) Find the total number of sales orders.

```
SELECT COUNT(SalesOrderID) AS Total_Orders
    FROM Sales.SalesOrderHeader;
```

4) Find the total sales amount (sum of SubTotal) from all orders.

```
SELECT SUM(SubTotal) AS Total_Sales
    FROM Sales.SalesOrderHeader;
```

5) Find the average and maximum order amount.

```
SELECT
    AVG(SubTotal) AS Avg_Order_Value,
    MAX(SubTotal) AS Max_Order_Value
FROM Sales.SalesOrderHeader;
```

6) Count how many orders were placed each year using GROUP BY.

```
SELECT
    YEAR(OrderDate) AS Order_Year,
    COUNT(SalesOrderID) AS Total_Orders
FROM Sales.SalesOrderHeader
GROUP BY YEAR(OrderDate)
    ORDER BY Order_Year;
```

7) Display total number of order details and total sales amount from SalesOrderDetail.

```
SELECT
    COUNT(*) AS No_of_OrderLines,
    SUM(LineTotal) AS Total_Sales
    FROM Sales.SalesOrderDetail;
```

Conclusion:

Aggregate functions such as **AVG, COUNT, MAX, MIN, SUM** are used to summarize and analyse data effectively.
Using the **AdventureWorks2019** database, these functions help to generate yearly, monthly, and overall summaries of sales data — providing valuable insights without the use of joins.